FIT5149 · Code-reading cheat sheet (Weeks 1–12)

Dùng để đọc code và dự đoán output/kết quả trong bài thi. Code bằng tiếng Anh; giải thích tiếng Việt + thuật ngữ tiếng Anh. Thư viện: scikit-learn, statsmodels/patsy, SciPy, PyTorch, pgmpy, pytorch-crf.

W1 · EDA & preprocessing (Ames housing, linear regression)

```
# dtypes / thống kê / số NaN mỗi cột
df.info(); df.describe(); df.isna().sum()
# numeric -> mean (median nếu lệch/outlier)
df[c].fillna(df[c].mean())
# categorical; mode() trả Series ĐÃ SẮP XẾP (hoà -> [0] = nhỏ
# nhất/alphabet)
df[c].fillna(df[c].mode()[0])
# k mức -> k-1 cột (tránh dummy-variable trap); tên cột Type_B
X = pd.get_dummies(df, drop_first=True)
# |r| cao giữa 2 feature => đa cộng tuyến; chọn feature tương
# quan với y
df.corr(); X.corrwith(y)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
random_state=0)
# fit CHỈ trên train (tránh leakage)
sc = StandardScaler(); Ztr = sc.fit_transform(Xtr); Zte =
sc.transform(Xte)
s = cross_val_score(LinearRegression(), X, y, cv=5,
scoring='neg_mean_squared_error')
# báo cáo mean ± std qua các fold
rmse = np.sqrt(-s); rmse.mean(), rmse.std()
```

- **ddof**: pandas `.std()` chia *n−1*; `np.std`, `StandardScaler` chia *n* (ddof=0). Ví dụ 3 giá trị 3,4,5: np.std=0.8165, ddof=1 → 1.0.
- `neg_mean_squared_error`: sklearn "càng lớn càng tốt" ⇒ trả giá trị **âm**; luôn đổi dấu `-s` rồi √. Mean của RMSE từng fold ≤ √(mean MSE) (Jensen), hai số **không bằng nhau**.
- MSE = mean(e^2); RMSE cùng đơn vị y; $R^2 = 1 - RSS/TSS$. Cột có NaN là float. Outlier: $|z|>3$ hoặc IQR; target lệch → `np.log`. Scale không đổi dự đoán của OLS nhưng đổi hệ số.

W2 · Model selection, regularisation, logistic regression, resampling

```
# lcv.alpha_, lcv.coef_, lcv.mse_path_ (n_alphas x n_folds)
lcv = LassoCV(cv=5, alphas=np.logspace(-3, 1, 50)).fit(X, y)
# chỉ số biến được giữ
np.nonzero(lcv.coef_)[0]
# logistic: res.params, res.summary()
res = sm.GLM(y, sm.add_constant(X), family=sm.families.Binomial()).fit()
# predict() = XÁC SUẤT; ngưỡng ">" là strict
prob = res.predict(sm.add_constant(X)); pred = (prob > 0.5).astype(int)
# cm = [[TN, FP], [FN, TP]]: HÀNG = true, CỘT = pred
err = np.mean(pred != y); cm = confusion_matrix(y, pred)
# bootstrap: lấy mẫu CÓ hoàn lại, kích thước n
b = resample(df)
se = np.std([boot_fn(resample(df)) for _ in range(B)]);
np.percentile(est, [2.5, 97.5])
```

- **Subset selection**: best subset = 2^p mô hình; forward/backward stepwise = $1 + p(p+1)/2$. RSS/R^2 luôn giảm/tăng khi thêm biến ⇒ chọn *kích thước* bằng CV hoặc C_p/AIC/BIC (nhỏ nhất), adj-R^2 (lớn nhất). `SequentialFeatureSelector(LinearRegression(), k_features=5, forward=True, scoring=..., cv=5).fit(X,y)` (mlxtend).
- **Ridge** `Ridge(alpha)`: min $\|y-Xw\|^2 + \alpha\|w\|^2$ (co hệ số, không bao giờ = 0). **Lasso** `Lasso(alpha)`: min $\frac{1}{2n}\|y-Xw\|^2 + \alpha\|w\|_1$ (một số hệ số đúng bằng 0 ⇒ feature selection). α↑ ⇒ variance↓, bias↑. Phải **standardise** trước.
- Khi $X^TX = nI$: Lasso $\hat{w}_j = \text{sign}(\hat{\beta}_j)\cdot\max(|\hat{\beta}_j|-\alpha, 0)$ (soft-threshold); Ridge $\hat{w}_j = \hat{\beta}_j \cdot n/(n+\alpha)$ (co đều).
- Logistic: log-odds = $\beta_0+\beta\cdot x$ tuyến tính; $p = 1/(1+e^{-(\beta_0+\beta\cdot x)})$. precision = TP/(TP+FP); recall = TP/(TP+FN); specificity = TN/(TN+FP). Hạ ngưỡng ⇒ recall↑, precision↓. Dữ liệu mất cân bằng (≈3% default): luôn đoán "No" đã có error ≈ 3% ⇒ cần so với baseline.
- **Validation set**: 1 lần chia, kết quả đổi theo split. **K-fold** (K=5/10) cân bằng bias–variance. **LOOCV** K=n, ít bias nhưng đắt/variance cao. **Bootstrap** SE = độ lệch chuẩn của ước lượng qua B mẫu; ~63.2% quan sát khác nhau trong mỗi mẫu.

W3 · Non-linear methods (polynomials, splines)

```
# [[1, 2, 3, 4, 6, 9]] = [1, a, b, a², ab, b²]; #cột = C(p+d,
# d)
PolynomialFeatures(degree=2).fit_transform([[2, 3]])
# [[2, 3, 6]] (bỏ a², b²)
PolynomialFeatures(2, include_bias=False,
interaction_only=True).fit_transform([[2, 3]])
# patsy: cột = degree + K  (+1 intercept) -> shape (n, 6)
dmatrix("bs(x, knots=(5, 7), degree=3)", {"x": x})
# df = degree + K  => K = 3 knots (quantile); shape (n, 7) gồm
# intercept
dmatrix("bs(x, df=6, degree=3)", {"x": x})
# degree=0: hằng số từng đoạn; params = [mean đoạn 1, Δ đoạn
# 2, Δ đoạn 3]
smf.ols("y ~ bs(x, knots=(5, 7), degree=0)", df).fit()
# natural spline (ISLP/patsy): tuyến tính ngoài biên
ns(x, df=4)
cross_val_score(LinearRegression(), Xpoly, y, cv=5,
scoring='neg_mean_squared_error')
```

- **Polynomial**: bậc d lớn ⇒ dao động mạnh ở biên (variance cao); chọn d bằng CV. **Step function**: `pd.cut`, hệ số = chênh lệch so với vùng cơ sở.
- **Regression spline**: cubic từng đoạn, liên tục f, f′, f″ tại K knots ⇒ 4(K+1) − 3K = **K+4** tham số (gồm intercept). **Natural spline**: tuyến tính ngoài hai boundary knot ⇒ variance thấp ở biên. Knots đặt theo quantile của x.
- **Smoothing spline**: min $\Sigma(y-g(x))^2 + \lambda\int g''(t)^2dt$. λ→0: nội suy (overfit); λ→∞: đường thẳng. Chọn λ bằng LOOCV; *effective df* = trace(S_λ). **GAM**: $f = \beta_0 + \Sigma f_j(x_j)$ — phi tuyến nhưng cộng tính, diễn giải được.

W4 · Support vector machines

```
# C rất lớn ~ hard margin
m = SVC(kernel='linear', C=1e6).fit(X, y)
# thứ tự alphabet, vd ['blue' 'red']; decision_function > 0
# <=>  classes_[1]
m.classes_
# số SV THEO TỪNG CLASS (theo classes_);  m.support_ = chỉ số
# (0-based);  m.support_vectors_
m.n_support_
# chỉ kernel linear; w = coef_[0]; bề rộng margin =
# 2/np.linalg.norm(w)
m.coef_, m.intercept_
# w·x+b ; +1/-1 = nằm đúng trên biên margin
m.decision_function([[3, 2]])
# accuracy; gọi trên dữ liệu train => TRAIN accuracy
m.score(X, y)
GridSearchCV(SVC(), {'C': [.1, 1, 10], 'gamma': [.01, .1, 1]},
cv=5).fit(X, y).best_params_
make_pipeline(StandardScaler(), SVC(kernel='rbf'))
# tf-idf = tf·idf; CountVectorizer = bag-of-words; fit CHỈ
# trên train
TfidfVectorizer().fit_transform(texts)
classification_report(yte, pred);
ConfusionMatrixDisplay.from_predictions(yte, pred)
```

- Siêu phẳng $w \cdot x + b = 0$, phân loại theo dấu. Margin: khoảng cách tới điểm gần nhất = $1/\|w\|$; **bề rộng toàn phần = $2/\|w\|$** khi $y(w \cdot x + b) \geq 1$ (dấu "=" tại support vector). Hard-margin chỉ khi tách được. Soft-margin: $\min \tfrac{1}{2}\|w\|^2 + C\Sigma\xi_i$; hinge loss $\max(0, 1 - y \cdot f(x))$.
- sklearn: **C lớn** ⇒ ít vi phạm, margin hẹp, ÍT support vector, bias thấp/variance cao (ngược với "budget C" của ISLR). **Chi support vector** (trên/trong margin hoặc phân sai) quyết định ranh giới; dời điểm khác không đổi mô hình.
- Kernel: linear $x \cdot x'$; poly $(\gamma x \cdot x' + r)^d$; **RBF** $\exp(-\gamma\|x - x'\|^2)$: γ lớn ⇒ cục bộ, dễ overfit; γ nhỏ ⇒ gần tuyến tính. Luôn **scale** feature trước SVM.
- Multi-class `SVC` dùng one-vs-one: $K(K-1)/2$ bộ phân loại. Đo trên dữ liệu mất cân bằng: dùng stratified CV, F1/Cohen κ thay vì accuracy.

W5 · Neural networks: MLP, CNN, attention/Transformer, LLM

```
# logits THÔ (chưa softmax) + chỉ số lớp; softmax 2 lần = sai
loss = nn.CrossEntropyLoss()(logits, target)
# backward() chỉ TÍNH grad và CỘNG DỒN vào .grad; step() mới
# cập nhật
opt.zero_grad(); loss.backward(); opt.step()
# dropout/BN khác nhau; no_grad khi suy luận
model.train() / model.eval();  with torch.no_grad(): ...
# H_out = floor((H + 2p - k)/s) + 1 ; params = 16*(3*5*5) + 16
# = 1216
nn.Conv2d(3, 16, kernel_size=5, stride=2, padding=2)
# (H,W)->(H/2,W/2) ;  nn.AdaptiveAvgPool2d(1) -> (B,C,1,1) ;
# nn.Flatten() -> (B, C*H*W)
nn.MaxPool2d(2)
# params V*d ; token id -> vector
nn.Embedding(V, d)
# mỗi HÀNG attn sum = 1
attn = torch.softmax(Q @ K.transpose(-2, -1) / math.sqrt(d_k), dim=-1) @
V
# causal mask: vị trí bị chặn có trọng số 0 sau softmax
scores.masked_fill(mask == 0, float('-inf'))
```

- Layer: **$A = g(XW + b)$**; $X(8,10) \cdot W(10,4) \to (8,4)$. **nn.Linear(in,out)** có in·out + out tham số. ReLU = $\max(0,u)$, đạo hàm $1[u>0]$. Output: regression = tuyến tính; binary = sigmoid; K lớp = softmax (trừ max để ổn định). Loss: MSE; BCE; **CE = $-\log p_{\text{đúng}}$**; hinge cho SVM.
- Backprop (softmax+CE, 1 mẫu): $dz = p - y$; $dW_2 = dz \cdot h^T$; $dh = W_2^T dz$; $du = dh \odot 1[u>0]$; $dW_1 = du \cdot x^T$. Unit có $u \leq 0$ ⇒ gradient 0 ("dead"). Cập nhật: $w \leftarrow w - lr \cdot \partial L/\partial w$.
- **CNN**: weight sharing + local connectivity + pooling (translation invariance). Số tham số conv không phụ thuộc kích thước ảnh. "same" padding: $p=(k-1)/2$ với stride 1.
- **Attention** = $\text{softmax}(QK^T/\sqrt{d_k})V$; chia $\sqrt{d_k}$ làm scores *nhỏ đi* để softmax không bão hoà. Multi-head: h đầu, mỗi đầu d/h chiều rồi concat + W_o. Positional encoding (sin/cos) vì attention không biết thứ tự. Block: self-attention → residual + LayerNorm → MLP → residual + LayerNorm. Chi phí $O(N^2 d)$.
- **ViT**: patchify ảnh H×W×C với patch P ⇒ $N=(H/P)(W/P)$ token, mỗi token P^2C chiều → Linear → +positional encoding → các block → head. CNN có inductive bias (locality) ⇒ tốt khi ít dữ liệu; ViT cần nhiều dữ liệu. **LLM**: BPE tokenisation (subword) → embedding → decoder-only Transformer (causal mask) pretrain dự đoán token kế tiếp → alignment (instruction tuning/RLHF).

W6 · Clustering: K-means, hierarchical, clustering learned representations

```
# km.labels_, km.cluster_centers_, km.inertia_,
# km.predict(Xnew)
km = KMeans(n_clusters=2, n_init=10, random_state=0).fit(X)
# vector rút gọn độ dài n(n-1)/2 ;  squareform(d) <-> ma trận
# vuông n×n
d = pdist(X)
# 'single' | 'complete' | 'average' | 'ward'.
# linkage(squareform-matrix) = BUG (bị coi là n quan sát p=n
# đặc trưng)
Z = linkage(d, method='single')
# cụm mới thứ i có id n+i ; height không giảm
# (single/complete/average/ward)
Z[i] = [idx_a, idx_b, height, size]
fcluster(Z, t=2, criterion='maxclust'); fcluster(Z, t=3.0,
criterion='distance'); dendrogram(Z)
# 512-d embedding -> normalize -> KMeans
feat = resnet18(weights=...); feat.fc = nn.Identity()
# ARI: 1 = khớp, ≈0 = ngẫu nhiên
adjusted_rand_score(true_labels, km.labels_); pd.crosstab(true,
km.labels_)
```

- **K-means** (Lloyd): lặp (a) tính centroid = mean của cụm, (b) gán điểm về centroid gần nhất. Mục tiêu **inertia** = $\Sigma\|x - \mu_k\|^2$ giảm sau mỗi bước nhưng chỉ tới **local optimum** ⇒ `n_init` chạy nhiều khởi tạo, không đảm bảo global. Inertia luôn giảm khi K tăng (K=n ⇒ 0) ⇒ chọn K bằng elbow/silhouette. **Scale** biến trước khi tính khoảng cách. Điểm mới → gán về centroid gần nhất.
- **Linkage**: *single* = min khoảng cách giữa 2 cụm (chaining, cụm dài); *complete* = max (cụm gọn); *average* = trung bình mọi cặp $(d(AB,C) = (|A|d(A,C)+|B|d(B,C))/(|A|+|B|))$; *ward* = tối thiểu tăng within-cluster variance. Thuật toán: gộp cặp có dissimilarity nhỏ nhất, cập nhật ma trận, lặp; vẽ dendrogram, chiều cao = mức dissimilarity khi gộp; cắt ở độ cao h hoặc K cụm.
- Cách làm tay (đề mẫu Oct-2023): liệt kê ma trận sau mỗi bước, vd single: $d((B,C),A) = \min(d(A,B),d(A,C))$. Cụm học từ **representation** (embedding CNN) tốt hơn pixel thô; ViT lab: so sánh CNN vs MiniViT, optimiser SGD vs Adam, đếm tham số, patchify.

W7 · Dimension reduction: PCA, PCR, t-SNE

```
# Xs = StandardScaler().fit_transform(X)   (PCA phụ thuộc
# scale!)
pca = PCA(n_components=2).fit(Xs)
# (k, p): MỖI HÀNG = 1 PC (loading, ‖φ‖=1) ;
# pca.explained_variance_ = eigenvalue λ (ddof=n-1)
pca.components_
# PVE tích luỹ ;  PCA(n_components=0.95) -> số PC nhỏ nhất đạt
# ≥ 95% phương sai
pca.explained_variance_ratio_.cumsum()
# score ;  pca.inverse_transform(Z) = tái tạo
Z = pca.transform(Xs)  == (Xs - pca.mean_) @ pca.components_.T
# PCR; chọn M bằng CV
pcr = Pipeline([('sc', StandardScaler()), ('pca', PCA(n_components=M)),
('lr', LinearRegression())])
# KHÔNG có .transform cho điểm mới
TSNE(n_components=2, perplexity=30, random_state=0).fit_transform(X)
# eigh trả eigenvalue TĂNG dần -> phải sắp giảm dần
S = Xc.T @ Xc / (n-1); vals, vecs = np.linalg.eigh(S)
```

- **Curse of dimensionality**: số vùng tăng mũ theo p; khi p≳n OLS khớp hoàn hảo train ($R^2 \to 1$, test tệ). PCA: PC_1 = hướng phương sai lớn nhất, các PC trực giao; $\lambda_m/\Sigma\lambda$ = PVE; tổng bình phương sai số tái tạo khi bỏ PC = $(n-1)\cdot\Sigma\lambda_{bỏ}$. Dấu eigenvector tuỳ ý. Hai cột không tương quan ⇒ PC trùng trục (tỉ lệ = tỉ lệ phương sai).
- **PCR** = PCA (unsupervised) rồi hồi quy trên M score; không bảo đảm các PC dự đoán tốt y. **TruncatedSVD**: không centre, dùng cho ma trận thưa (bag-of-words).
- **t-SNE**: giữ láng giềng cục bộ (P Gaussian ở chiều cao, Q Student-t ở chiều thấp, tối thiểu $KL(P\|Q)$); *perplexity* ≈ số láng giềng hiệu dụng (5–50); ngẫu nhiên ⇒ đặt random_state; khoảng cách giữa cụm & kích thước cụm không có ý nghĩa; thường PCA→50 chiều trước. PCA tuyến tính, giữ cấu trúc toàn cục, có transform.

W8 · Bayesian networks (pgmpy)

```
bn = DiscreteBayesianNetwork([('D','G'), ('I','G'), ('I','S'),
('G','L')])
TabularCPD('G', 3, values, evidence=['I','D'], evidence_card=[2,2],
state_names={...})
# values.shape = (card(G), prod card(parents)) ; mỗi CỘT tổng
# = 1 ; BIẾN evidence CUỐI đổi nhanh nhất:
# cột = (I0,D0), (I0,D1), (I1,D0), (I1,D1) khi
# evidence=['I','D']
# True nếu CPD khớp graph (cardinality, cột tổng 1) else
# ValueError
bn.add_cpds(c1, c2, ...); bn.check_model()
# d-connected? (True = KHÔNG bị chặn)
bn.is_dconnected('D','I'); bn.is_dconnected('D','I', observed=['L'])
bn.get_markov_blanket('G'); bn.local_independencies('G');
bn.get_independencies()
bn.get_parents('G'); bn.get_children('G'); bn.get_roots();
bn.get_leaves(); bn.get_immoralities()
# forward sample ;  bn.fit(df, estimator=...)  # học tham số
bn.simulate(100)
```

- **Factorisation**: $P(X_1..X_n) = \Pi\, P(X_i \mid Pa(X_i))$. Student: $P(d,i,g,s,l)=P(d)P(i)P(g|i,d)P(s|i)P(l|g)$. **#tham số độc lập** = $\Sigma_i (|X_i|-1)\cdot\Pi|Pa_i|$ (G 3 trạng thái, 2 cha nhị phân: 2×4 = 8, không phải 12); bảng joint đầy đủ: $\Pi|X_i| - 1$.
- **d-separation** (path giữa A,B): *chain* A→M→B và *fork* A←M→B: **bị chặn khi M được quan sát**; *collider* A→M←B: bị chặn khi M **và mọi hậu duệ của M** KHÔNG được quan sát (quan sát M hoặc hậu duệ ⇒ mở = *explaining away*). Hai nút d-separated ⇔ mọi path bị chặn; chỉ cần một path mở ⇒ d-connected. D-sep ⇒ độc lập có điều kiện.
- **Markov blanket** = cha + con + cha khác của các con. Loại lập luận: causal (top-down), evidential (bottom-up), intercausal. Học tham số từ đếm; thiết kế BN từ mô tả: nguyên nhân → hệ quả, mediator, confounder (C→T, C→Y). Deep BN (pomegranate): encoder NN biến input thô (ảnh) thành biến rời rạc làm nút trong BN; dữ liệu thiếu dùng `MaskedTensor`.

W9 · Markov networks, factor graphs, CRF

```
mn = DiscreteMarkovNetwork(); mn.add_edges_from([('A','B'), ('B','C')])
# values (A0B0, A0B1, A1B0, A1B1): biến ĐẦU đổi chậm nhất
f1 = DiscreteFactor(['A','B'], [2,2], [3,1,1,3])
# tích factor: scope = hợp, 3 biến -> 8 ô
mn.add_factors(f1, f2); psi = f1 * f2
# sum-out / điều kiện hoá / chuẩn hoá
psi.marginalize(['B'], inplace=False); f1.reduce([('A',0)],
inplace=False); tau.normalize()
# THỨ TỰ scope() của pgmpy không ổn định -> luôn tự sort khi
# so sánh
sorted(psi.scope())
BeliefPropagation(mn).query(['C'], evidence={'A':0});
mn.get_local_independencies(); bn.to_markov_model();
mn.to_factor_graph()
crf = CRF(n_tags, batch_first=True)
# crf(...) trả LOG-LIKELIHOOD ⇒ loss = âm của nó
nll = -crf(emissions, tags, mask=mask, reduction='mean')
# Viterbi: list các chuỗi tag tốt nhất ;  crf.transitions[i,
# j] = điểm chuyển tag i -> j
crf.decode(emissions, mask=mask)
```

- **Gibbs distribution**: $P(x) = (1/Z)\,\Pi_c\,\varphi_c(x_c)$, φ ≥ 0 trên *clique*; $\mathbf{Z = \Sigma_X \Pi\varphi}$ (partition function; vd Z=24 ⇒ chia 24 mới ra xác suất). Factor là điểm tương thích, không phải xác suất. Hammersley–Clifford: P>0 thoả các độc lập của đồ thị ⇔ phân rã theo clique cực đại.
- **Độc lập trong MN** = *graph separation*: $X \perp Y|Z$ nếu Z chặn mọi path; Markov blanket = hàng xóm. Đồ thị liên thông ⇒ không có độc lập biên ($A \perp E$ không suy ra được). **BN → MN** = *moralisation* (nối các cha chung của một con, bỏ hướng); factor = CPD ⇒ Z=1. **Factor graph**: đồ thị hai phía biến–factor.
- **Linear-chain CRF**: $P(y|x) = (1/Z(x))\cdot\exp(\Sigma_t\,\text{emit}(t,y_t) + \Sigma_t\,\text{trans}(y_{t-1},y_t))$; Z(x) tính bằng forward algorithm $O(T\cdot K^2)$ thay vì liệt kê K^T chuỗi; decode bằng Viterbi (MAP). Softmax per-token chọn nhãn độc lập ⇒ có thể ra chuỗi BIO không hợp lệ (O → I-X); CRF học **transition** (so learned với empirical; ô không hợp lệ rất âm) ⇒ nhất quán cấp chuỗi. Metric: token accuracy dễ cao giả; dùng span F1 / Macro F1.

W10 · Exact inference: variable elimination, MAP

```
ve = VariableElimination(bn)
# DiscreteFactor đã CHUẨN HOÁ
ve.query(['Cancer'], evidence={'Smoker':'T'},
elimination_order='MinFill', show_progress=False)
# values thứ tự theo state_names;  joint=True (mặc định) ->
# bảng joint các biến query
q.values, q.state_names
# {'Cancer': 'F'}  MAP assignment (dict)
ve.map_query(['Cancer'], evidence={...})
BeliefPropagation(bn).query([...]);
bn.to_markov_model().to_factor_graph()
```

- **Queries**: marginal P(X); conditional P(Q|e) = P(Q,e)/P(e); **MAP** = argmax_q P(q|e) (sum-product → *max-product*: thay Σ bằng max, rồi truy ngược).
- **Variable elimination**: (1) chuyển BN→MN→factor graph, mỗi CPD là một factor; (2) **reduce** các factor chứa evidence (giữ ô khớp); (3) chọn thứ tự loại biến (biến evidence/query không bị loại); (4) với mỗi biến Z theo thứ tự: ψ = Π (mọi factor chứa Z), τ = Σ_Z ψ, thay vào danh sách; (5) nhân các factor còn lại, **chuẩn hoá** → P(Q|e). Chain A→B→C→D: τ_1 = pA·P(B|A) = P(B), ... ⇒ n·k^2 thay cho k^n.
- **Chi phí** ∝ Σ $k^{|\text{scope}(\psi)|}$, scope = biến loại + hàng xóm hiện tại ⇒ lớn nhất = *induced width*+1. Kết quả <u>không đổi</u> theo thứ tự, chỉ chi phí đổi; tìm thứ tự tối ưu là NP-khó ⇒ heuristic **MinFill** (ít cạnh bù nhất), **MinNeighbors**, **MinWeight**, **WeightedMinFill**. Cây/chuỗi ⇒ width nhỏ ⇒ chính xác & rẻ; lưới/đồ thị dày ⇒ cần xấp xỉ (sampling). Belief propagation (junction tree) tính mọi marginal một lượt.

W11 · Approximate inference: sampling

```
# lấy mẫu rời rạc: lớp ĐẦU TIÊN có u ≤ cumulative (xác suất)
cum = np.cumsum(p); k = np.argmax(u <= cum)
# rejection (liên tục): z0~q ;  u0~U(0, k*q(z0)) ;  nhận z0
# nếu u0 < p~(z0) ;  cần k*q(z) >= p~(z) mọi z ; acceptance =
# ∫p~ / k (q chuẩn hoá)
s = BayesianModelSampling(bn)
# ancestral: theo thứ tự topo, mỗi biến ~ P(X | cha đã lấy
# mẫu)
s.forward_sample(size=1000, seed=0)
# forward + bỏ mẫu không khớp evidence
s.rejection_sample(evidence=[State('Smoker','T')], size=1000)
# evidence cố định, mẫu có trọng số
s.likelihood_weighted_sample(evidence=[...], size=1000)
ApproxInference(bn).query(['Cancer'], n_samples=10000, evidence={...})
# pgmpy: không nhận evidence; notebook dùng pyAgrum
# gum.GibbsSampling
GibbsSampling(model).sample(start_state=None, size=1000)
```

- **Forward sampling**: thứ tự <u>không duy nhất</u> (mọi thứ tự topological). **Rejection**: tỉ lệ nhận ≈ P(e) ⇒ lãng phí khi evidence hiếm. **Gibbs (MCMC)**: giữ evidence cố định; lặp, mỗi lần lấy lại một biến X_i ~ P(X_i | MB(X_i)) ∝ Π factor chứa X_i (các nhân tử không chứa X_i triệt tiêu); mẫu *tự tương quan* ⇒ cần **burn-in**, nhiều sweep; hội tụ về phân phối dừng P(X|e). Ước lượng E[f] ≈ (1/N)Σf($x^{(s)}$); sai số Monte Carlo ∝ 1/√N.
- So sánh: forward/rejection cho mẫu *độc lập* nhưng bỏ phí; Gibbs không bỏ phí nhưng phụ thuộc, mixing chậm. **N-gram language model** là một BN (P(w_t | w_{t-1}, w_{t-2})): tham số bằng MLE từ đếm Counter; sinh câu bằng lấy mẫu từ kế tiếp; Gibbs resample từng vị trí.

W12 · Structure learning (PC algorithm), parameter estimation

```
# CI test (χ²): p ≥ α  => không bác bỏ H0 "độc lập" => X ⊥ Y |
# Z
chi_square(X, Y, Z, data, boolean=True)
# 'skeleton' | 'pdag' | 'dag'
PC(data).estimate(ci_test='chi_square', significance_level=0.05,
return_type='dag')
# score-based: thêm/xoá/đảo cạnh; 'bic-d' cần cột dtype
# category (int bị coi là liên tục)
HillClimbSearch(data).estimate(scoring_method='bic-d')
# MLE: θ(x|pa) = N(x,pa)/N(pa)  (đếm)
bn.fit(df, estimator=DiscreteMLE())
# +1 pseudo-count mỗi ô
bn.fit(df, estimator=DiscreteBayesianEstimator(prior_type='K2'))
# mặc định BDeu, ESS=5
DiscreteBayesianEstimator(prior_type='BDeu', equivalent_sample_size=5)
```

- **PC**: (1) **skeleton**: bắt đầu đồ thị đầy đủ; với |U|=0,1,2,...: xoá cạnh X–Y nếu có U (từ láng giềng) sao cho X⊥Y|U (p ≥ α) và ghi *sepset*(X,Y)=U; (2) **immorality**: với X–Z–Y, X,Y không kề: là collider X→Z←Y ⇔ **Z ∉ sepset(X,Y)**; (3) **Meek rules**: R1 a→b–c, a,c không kề ⇒ b→c (tránh immorality mới); R2 a→b→c, a–c ⇒ a→c (tránh chu trình); R3. Kết quả là **CPDAG**, không phải DAG. α nhỏ (0.01) giữ ít cạnh ngẫu nhiên hơn α=0.05; kiểm định nhạy với nhiễu/ít dữ liệu. Độ phức tạp O(n^{d+2}).
- **I-equivalence**: hai DAG tương đương ⇔ <u>cùng skeleton VÀ cùng tập immoralities</u> (v-structure mà hai cha không kề nhau). X→Y→Z, X←Y←Z, X←Y→Z tương đương (X⊥Z|Y) ⇒ PC trả X–Y–Z; X→Y←Z (X⊥Z nhưng X⊥̸Z|Y) là duy nhất. Cùng skeleton chưa đủ. P-map duy nhất tới I-equivalence.
- **MLE** cho xác suất 0 với tổ hợp chưa gặp ⇒ log-likelihood −∞; prior Dirichlet (K2/BDeu) = làm mượt. Số mẫu nhỏ ⇒ ước lượng nhiễu. **Score** BIC = log L − (d/2)·log N (phạt độ phức tạp); phân rã theo nút ⇒ cập nhật cục bộ khi thêm/xoá cạnh.

Mẫu tính tay & số cần nhớ (hay gặp trong đề)

- **Số cần nhớ**: e^{-1}=0.368, e^{-2}=0.135, e^{1}=2.718, e^{2}=7.389, e^{6}=403; σ(0)=0.5, σ(1)=0.731, σ(2)=0.881, σ(3)=0.953; ln2=0.693, ln3=1.099, ln10=2.303; √2=1.414, √3=1.732.
- **Softmax + CE**: z=[2,0] ⇒ p=[0.881,0.119], loss=−ln0.881=**0.127**; softmax 2 lần ⇒ p=[0.88,0.12] đưa vào CE ⇒ 0.383 (sai). z=[−1.5,4.5], nhãn lớp 0 ⇒ p=[0.002,0.998], loss=ln(1+e^{6})=6.002, argmax=1 ⇒ phân loại sai.
- **Bayes từ bảng đếm** (đề mẫu BN): P(Y=1|C=long)=Σ_TP(Y=1|T,C)P(T|C)=0.38·(1/7)+0.21·(6/7)=164/700≈0.234; P(T=A|C=long,Y=1)=0.38·(1/7)/0.234≈0.232 (prior P(T|C) lấn át likelihood).
- **Single linkage tay** (đề mẫu Oct-2023): gộp cặp nhỏ nhất (B,C)@0.2; d((BC),A)=min(0.5,0.7)=0.5; d((BC),D)=min(0.9,0.7)=0.7 ⇒ gộp A@0.5, rồi D@0.7, cuối cùng E@1.0. Average linkage: lấy trung bình mọi cặp (vd (6+5)/2=5.5).
- **PCA 2×2**: S=[[a,b],[b,a]] ⇒ λ=a±b, eigenvector (1,±1)/√2; PVE_1=(a+|b|)/(2a). Hai cột không tương quan ⇒ PC trùng trục.
- **Lasso/Ridge tay** (X^TX=nI, $\hat{\beta}$=[4,−3,2,1,0.5], α=1.5): Lasso ⇒ [2.5,−1.5,0.5,0,0] (2 hệ số = 0); Ridge n=8,α=8 ⇒ $\hat{\beta}$=8/16=[2,−1.5,1,0.5,0.25] (0 hệ số = 0).
- **SVM tay**: SV (2,2),(4,4)∈red; (2,1),(4,3)∈blue ⇒ w=(−2,2), b=1: f(x)=$-2x_1+2x_2+1$; margin=2/‖w‖=0.707; f(3,2)=−1 (nằm đúng biên phía blue).
- **Đếm tham số**: MLP 2→3→2: 2·3+3+3·2+2=17; Conv2d(3,16,5): 16·75+16=1216; Linear(4096,10): 40 970; BN (5 biến) với |D|=2,|I|=2,|G|=3,|S|=2,|L|=2: 1+1+8+2+3=15 vs joint 47.
- **CRF/VE**: Z (T=3, K=2) = 8 chuỗi ⇒ cộng 8 điểm; hoặc loại Y_3 rồi Y_2: $\tau_3(y_2)$=Σtrans·$emit_3$, $\tau_2(y_1)$=Σtrans·$emit_2$·τ_3, Z=Σ$emit_1$·τ_2. Xác suất nhãn giữa = (tổng điểm các chuỗi có nhãn đó)/Z.
- **d-separation nhanh**: chain/fork: mở khi M <u>chưa</u> quan sát; collider: mở khi M <u>hoặc hậu duệ</u> được quan sát. Student (D→G←I, I→S, G→L): D,I d-separated; quan sát L ⇒ d-connected; D,S d-connected | G; L,S d-separated | G (chain bị chặn).
- **BN đề mẫu (Dr Johnson)**: C→T, C→Y, T→Y, T→M, M→Y ⇒ P(C)P(T|C)P(M|T)P(Y|C,T,M). C,M d-connected qua C→T→M; quan sát T ⇒ C⊥M|T (collider Y chưa quan sát chặn hai path còn lại). Moralise: cha của Y là {C,T,M}; C–T và T–M đã có cạnh, chỉ cần thêm **C–M**, rồi bỏ hướng.
- **MN đề mẫu (Alice…Emma)**: cạnh A–B, B–C, B–D, C–D, B–E, D–E; clique cực đại {A,B}, {B,C,D}, {B,D,E} ⇒ P=(1/Z)φ(A,B)φ(B,C,D)φ(B,D,E). A⊥{C,D,E}|B đúng; C⊥E|B <u>sai</u> (còn path C–D–E); C⊥{A,E}|{B,D} đúng.
- **PC tay** (α=0.05): p(X,Y)=0.70 ⇒ xoá X–Y, sepset=ø; p(W,X|Z)=0.60, p(W,Y|Z)=0.50 ⇒ xoá, sepset={Z}. Skeleton: W–Z, X–Z, Y–Z. X–Z–Y: Z∉ø ⇒ collider X→Z←Y; Meek R1 ⇒ Z→W.
- **Rejection/forward sample**: so u với cumulative; nhận z_o nếu u·k·q(z_o) < $\tilde{p}$(z_o); acceptance lý thuyết = ∫$\tilde{p}$/k; rejection BN giữ ≈ N·P(e) mẫu.

Checklist khi đọc code trong phòng thi

- **Đọc đúng thứ được in**: shape vs giá trị, `.round()`, float/int, chuỗi hay mảng; đếm số dòng `print`.
- **Mặc định hay gây bẫy**: pandas std ddof=1 / numpy ddof=0; `neg_` scoring; confusion matrix hàng=true; ngưỡng `>` strict; `mode()[0]` khi hoà; `drop_first`; `n_support_` theo class; TSNE không có `transform`; `bias=True` trong Conv/Linear; `eigh` sắp tăng dần.
- **Tự kiểm tra**: hàng softmax/CPD cộng = 1; số tham số = Σ cột·hàng + bias; xác suất ∈ [0,1]; chuẩn hoá bằng tổng; sanity check bằng trường hợp biên (K=n, α→∞, λ→0).
- **pgmpy**: `TabularCPD` cột theo evidence cuối đổi nhanh nhất; `DiscreteFactor` biến đầu đổi chậm nhất; `query` trả factor đã chuẩn hoá; thứ tự `scope()` không ổn định.
- **Hướng làm**: nhận diện hàm & tham số mặc định → tính tay một phần nhỏ → đối chiếu từng lựa chọn → kết luận kèm lý do (đề mẫu chấm cả lập luận).